

Skill Session 15

Task 1

To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

```
j_rakshitha@RAKSHITHA: ~  
j_rakshitha@RAKSHITHA:~$ nano session15_task1_combined.c  
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra -g session15_task1_combined.c -o  
session15_task1_combined  
j_rakshitha@RAKSHITHA:~$ ./session15_task1_combined  
Session 15 - Task 1: Combined Redirection  
Example: echo Hello > output.txt 2> error.txt  
  
combined-shell$ echo Hello_OSSP > combined_output.txt 2> combined_error.txt  
Combined redirection completed successfully.  
j_rakshitha@RAKSHITHA:~$ cat combined_output.txt  
Hello_OSSP  
j_rakshitha@RAKSHITHA:~$ cat combined_error.txt  
j_rakshitha@RAKSHITHA:~$ ./session15_task1_combined  
Session 15 - Task 1: Combined Redirection  
Example: echo Hello > output.txt 2> error.txt  
  
combined-shell$ ls missing_file > combined_output.txt 2> combined_error.txt  
Combined redirection completed successfully.  
j_rakshitha@RAKSHITHA:~$ cat combined_error.txt  
ls: cannot access 'missing_file': No such file or directory  
j_rakshitha@RAKSHITHA:~$ cat combined_output.txt  
j_rakshitha@RAKSHITHA:~$ |
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session15_task1_combined.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    char command[512];

    printf("Session 15 - Task 1: Combined Redirection\n");
    printf("Example: echo Hello > output.txt 2> error.txt\n\n");

    printf("combined-shell$ ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
        return 1;

    command[strcspn(command, "\n")] = '\0';

    char *out_op = strstr(command, ">");
    char *err_op = strstr(command, "2>");

    if (out_op == NULL || err_op == NULL)
    {
        printf("Error: command must contain both > and 2>\n");
        return 1;
    }

    /* Make sure we use the correct positions */
    if (err_op < out_op)
    {
        printf("Error: use format command > output.txt 2> error.txt\n");
        return 1;
    }

    [ Read 159 lines ]
    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session15_task1_combined.c
}

/* Separate stderr filename */
*err_op = '\0';
char *error_file = err_op + 2;

/* Separate stdout filename */
*out_op = '\0';
char *output_file = out_op + 1;

/* Trim spaces */
while (*output_file == ' ' || *output_file == '\t')
    output_file++;

while (*error_file == ' ' || *error_file == '\t')
    error_file++;

char *end = output_file + strlen(output_file) - 1;
while (end >= output_file &&
    (*end == ' ' || *end == '\t'))
{
    *end = '\0';
    end--;
}

end = error_file + strlen(error_file) - 1;
while (end >= error_file &&
    (*end == ' ' || *end == '\t'))
{
    *end = '\0';
    end--;
}

char *program = command;

while (*program == ' ' || *program == '\t')
    program++;

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session15_task1_combined.c

if (strlen(program) == 0 ||
    strlen(output_file) == 0 ||
    strlen(error_file) == 0)
{
    printf("Error: invalid command or filenames.\n");
    return 1;
}

int out_fd = open(output_file,
                  O_WRONLY | O_CREAT | O_TRUNC,
                  0644);

if (out_fd == -1)
{
    perror("output file");
    return 1;
}

int err_fd = open(error_file,
                  O_WRONLY | O_CREAT | O_TRUNC,
                  0644);

if (err_fd == -1)
{
    perror("error file");
    close(out_fd);
    return 1;
}

pid_t pid = fork();

if (pid == -1)
{
    perror("fork");
    close(out_fd);
    close(err_fd);
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session15_task1_combined.c
}

if (pid == 0)
{
    if (dup2(out_fd, STDOUT_FILENO) == -1)
    {
        perror("dup2 stdout");
        exit(1);
    }

    if (dup2(err_fd, STDERR_FILENO) == -1)
    {
        perror("dup2 stderr");
        exit(1);
    }

    close(out_fd);
    close(err_fd);
}
```

```
char *args[50];
int count = 0;

char *token = strtok(program, " \t");

while (token != NULL && count < 49)
{
    args[count++] = token;
    token = strtok(NULL, " \t");
}

args[count] = NULL;

execvp(args[0], args);

perror("execvp");
exit(1);
}

close(out_fd);
close(err_fd);

waitpid(pid, NULL, 0);

printf("Combined redirection completed successfully.\n");

return 0;
}
|
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

Task 2

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

```
j_rakshitha@RAKSHITHA:~$ nano session15_task2_complex.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra -g session15_task2_complex.c -o session15_task2_complex
j_rakshitha@RAKSHITHA:~$ ./session15_task2_complex
Session 15 - Task 2: Pipes and Redirection
Example: ls -l | grep .c > result.txt
```

```
complex-shell$ ls -l | grep .c > result.txt
```

Execution Plan:

1. Execute first command: ls -l
2. Send its output through a pipe.
3. Execute second command: grep .c
4. Redirect final output to: result.txt

Complex command completed successfully.

```
j_rakshitha@RAKSHITHA:~$ cat result.txt
-rw-r--r-- 1 j_rakshitha j_rakshitha 60 Sep 17 08:00 combined_error.txt
-rw-r--r-- 1 j_rakshitha j_rakshitha 0 Sep 17 08:00 combined_output.txt
-rwxr-xr-x 1 j_rakshitha j_rakshitha 16360 Aug 3 05:51 command_executor
-rw-r--r-- 1 j_rakshitha j_rakshitha 981 Aug 3 05:47 command_executor.c
drwxr-xr-x 5 j_rakshitha j_rakshitha 4096 Aug 18 05:21 ossp_project
-rw-r--r-- 1 j_rakshitha j_rakshitha 1970 Sep 17 04:42 session10_export.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 963 Sep 17 04:40 session10_pwd_exit.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 2997 Sep 17 05:10 session11_history.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 2232 Sep 17 05:22 session11_pipeline.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 2576 Sep 17 05:27 session12_multi_pipe.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1560 Sep 17 05:24 session12_pipe.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1926 Sep 17 06:49 session13_task2_output.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 2655 Sep 17 06:58 session14_task1_append.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 2784 Sep 17 06:59 session14_task2_stderr.c
-rwxr-xr-x 1 j_rakshitha j_rakshitha 20880 Sep 17 07:58 session15_task1_combined
-rw-r--r-- 1 j_rakshitha j_rakshitha 3260 Sep 17 07:58 session15_task1_combined.c
-rwxr-xr-x 1 j_rakshitha j_rakshitha 21032 Sep 17 08:03 session15_task2_complex
-rw-r--r-- 1 j_rakshitha j_rakshitha 3533 Sep 17 08:03 session15_task2_complex.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 391 Aug 12 06:39 task1.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 933 Aug 12 06:41 task2.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1218 Aug 12 06:44 task3.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 3814 Aug 12 06:50 task4.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1786 Aug 12 06:52 task5.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1378 Aug 18 05:48 week4_T1.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 828 Aug 18 05:45 week4_T2.c
```

```
j_rakshitha@RAKSHITHA: ~ × + v
GNU nano 8.7.1 session15_task2_complex.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    char command[512];

    printf("Session 15 - Task 2: Pipes and Redirection\n");
    printf("Example: ls -l | grep .c > result.txt\n\n");

    printf("complex-shell$ ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
        return 1;

    command[strcspn(command, "\n")] = '\0';

    char *pipe_op = strchr(command, '|');
    char *redirect_op = strchr(command, '>');

    if (pipe_op == NULL || redirect_op == NULL)
    {
        printf("Error: expected both | and > operators.\n");
        return 1;
    }

    if (redirect_op < pipe_op)
    {
        printf("Error: expected pipe before output redirection.\n");
        return 1;
    }

    *pipe_op = '\0';
    *redirect_op = '\0';
}
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session15_task2_complex.c
char *cmd1 = command;
char *cmd2 = pipe_op + 1;
char *filename = redirect_op + 1;

while (*cmd1 == ' ' || *cmd1 == '\t')
    cmd1++;

while (*cmd2 == ' ' || *cmd2 == '\t')
    cmd2++;

while (*filename == ' ' || *filename == '\t')
    filename++;

char *end = filename + strlen(filename) - 1;

while (end >= filename &&
       (*end == ' ' || *end == '\t'))
{
    *end = '\0';
    end--;
}

if (strlen(cmd1) == 0 ||
    strlen(cmd2) == 0 ||
    strlen(filename) == 0)
{
    printf("Error: invalid command.\n");
    return 1;
}

printf("\nExecution Plan:\n");
printf("1. Execute first command: %s\n", cmd1);
printf("2. Send its output through a pipe.\n");
printf("3. Execute second command: %s\n", cmd2);
printf("4. Redirect final output to: %s\n\n", filename);

int pipefd[2];

if (pipe(pipefd) == -1)
{
    perror("pipe");
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session15_task2_complex.c
    }
    return 1;

int output_fd = open(filename,
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

if (output_fd == -1)
{
    perror("open");
    close(pipefd[0]);
    close(pipefd[1]);
    return 1;
}

pid_t pid1 = fork();

if (pid1 == -1)
{
    perror("fork");
    return 1;
}

if (pid1 == 0)
{
    dup2(pipefd[1], STDOUT_FILENO);

    close(pipefd[0]);
    close(pipefd[1]);
    close(output_fd);

    char *args[50];
    int count = 0;

    char *token = strtok(cmd1, " \t");

    while (token != NULL && count < 49)
    {
        args[count++] = token;
        token = strtok(NULL, " \t");
    }
}
```

```

        args[count] = NULL;

        execvp(args[0], args);

        perror("execvp");
        exit(1);
    }

    pid_t pid2 = fork();

    if (pid2 == -1)
    {
        perror("fork");
        return 1;
    }

    if (pid2 == 0)
    {
        dup2(pipefd[0], STDIN_FILENO);
        dup2(output_fd, STDOUT_FILENO);

        close(pipefd[0]);
        close(pipefd[1]);
        close(output_fd);

        char *args[50];
        int count = 0;

        char *token = strtok(cmd2, " \t");

        while (token != NULL && count < 49)
        {

```

```

        args[count++] = token;
        token = strtok(NULL, " \t");
    }

    args[count] = NULL;

    execvp(args[0], args);

    perror("execvp");
    exit(1);
}

close(pipefd[0]);
close(pipefd[1]);
close(output_fd);

waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("Complex command completed successfully.\n");

return 0;
}
|

```

Skill Session 16

Task 1

To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

```

j_rakshitha@RAKSHITHA:~$ nano session16_task1_background.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra -g session16_task1_background.c -o session16_task1_background
j_rakshitha@RAKSHITHA:~$ ./session16_task1_background
Session 16 - Task 1: Background Jobs
Example: sleep 5 &

background-shell$ sleep 5 &
[1] Background process started with PID 802
Prompt returned immediately.
Job status: RUNNING
j_rakshitha@RAKSHITHA:~$ |

```

```
j_rakshitha@RAKSHITHA: ~ × + v
GNU nano 8.7.1 session16_task1_background.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    char command[256];

    printf("Session 16 - Task 1: Background Jobs\n");
    printf("Example: sleep 5 &\n\n");

    printf("background-shell$ ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
        return 1;

    command[strcspn(command, "\n")] = '\0';

    int len = strlen(command);

    /* Check for & at the end */
    if (len == 0 || command[len - 1] != '&')
    {
        printf("Error: command must end with &\n");
        return 1;
    }

    command[len - 1] = '\0';

    /* Remove trailing spaces */
    len = strlen(command);

    while (len > 0 &&
           (command[len - 1] == ' ' ||
            command[len - 1] == '\t'))
    {
        command[len - 1] = '\0';
    }
}
```

[Read 101 lines]

```
j_rakshitha@RAKSHITHA: ~ × + v
GNU nano 8.7.1 session16_task1_background.c

    len--;
}

if (strlen(command) == 0)
{
    printf("Error: empty background command.\n");
    return 1;
}

pid_t pid = fork();

if (pid == -1)
{
    perror("fork");
    return 1;
}

if (pid == 0)
{
    char *args[50];
    int count = 0;

    char *token = strtok(command, " \t");

    while (token != NULL && count < 49)
    {
        args[count++] = token;
        token = strtok(NULL, " \t");
    }

    args[count] = NULL;

    execvp(args[0], args);

    perror("execvp");
    exit(1);
}

printf("[1] Background process started with PID %d\n", pid);
printf("Prompt returned immediately.\n");
```

```
/* Check whether the process has already completed */
int status;
pid_t result = waitpid(pid, &status, WNOHANG);

if (result == 0)
{
    printf("Job status: RUNNING\n");
}
else if (result == pid)
{
    printf("Job status: COMPLETED\n");
}
else
{
    perror("waitpid");
}

return 0;
}
```

Task 2

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

```
j_rakshitha@RAKSHITHA: ~  
j_rakshitha@RAKSHITHA:~$ nano session16_task2_jobtable.c  
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra -g session16_task2_jobtable.c -o  
session16_task2_jobtable  
j_rakshitha@RAKSHITHA:~$ ./session16_task2_jobtable  
Session 16 - Task 2: Job Table  
Commands:  
  run <command> - start background job  
  jobs          - display jobs  
  update        - update job states  
  cleanup       - remove completed jobs  
  exit          - quit  
  
job-shell$ run sleep 5  
[1] 1121 started  
job-shell$ jobs  
  
Job Table  
-----  
ID      PID      STATE      COMMAND  
1       1121     COMPLETED      sleep 5  
job-shell$ cleanup  
Completed jobs removed.  
job-shell$ jobs  
  
Job Table  
-----  
No jobs.  
job-shell$ run sleep 10  
[1] 1128 started  
job-shell$ jobs  
  
Job Table  
-----  
ID      PID      STATE      COMMAND  
1       1128     RUNNING      sleep 10  
job-shell$ update  
Job states updated.  
job-shell$ jobs
```

```
Job Table  
-----  
No jobs.  
job-shell$ exit  
  
Final Job Table:  
  
Job Table  
-----  
No jobs.  
j_rakshitha@RAKSHITHA:~$ |
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session16_task2_jobtable.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_JOBS 10

typedef enum
{
    RUNNING,
    COMPLETED
} JobState;

typedef struct
{
    int job_id;
    pid_t pid;
    char command[100];
    JobState state;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

void display_jobs(void)
{
    printf("\nJob Table\n");
    printf("-----\n");

    if (job_count == 0)
    {
        printf("No jobs.\n");
        return;
    }

    printf("ID\tPID\tSTATE\t\tCOMMAND\n");

    for (int i = 0; i < job_count; i++)
    {
        printf("%d\t%d\t%s\t\t%s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].state == RUNNING ? "RUNNING" : "COMPLETED",
            jobs[i].command);
    }
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session16_task2_jobtable.c
}
}

void update_jobs(void)
{
    int status;

    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].state == RUNNING)
        {
            pid_t result = waitpid(jobs[i].pid,
                                   &status,
                                   WNOHANG);

            if (result == jobs[i].pid)
            {
                jobs[i].state = COMPLETED;
            }
        }
    }
}

void remove_completed_jobs(void)
{
    for (int i = 0; i < job_count;)
    {
        if (jobs[i].state == COMPLETED)
        {
            for (int j = i; j < job_count - 1; j++)
            {
                jobs[j] = jobs[j + 1];
            }

            job_count--;
        }
        else
        {
            i++;
        }
    }
}

void launch_background(char *command)
{
    if (job_count >= MAX_JOBS)
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session16_task2_jobtable.c
{
    printf("Job table is full.\n");
    return;
}

pid_t pid = fork();
if (pid == -1)
{
    perror("fork");
    return;
}

if (pid == 0)
{
    char *args[50];
    int count = 0;

    char *token = strtok(command, " \\t");

    while (token != NULL && count < 49)
    {
        args[count++] = token;
        token = strtok(NULL, " \\t");
    }

    args[count] = NULL;

    execvp(args[0], args);

    perror("execvp");
    exit(1);
}

jobs[job_count].job_id = job_count + 1;
jobs[job_count].pid = pid;
jobs[job_count].state = RUNNING;

strncpy(jobs[job_count].command,
        command,
        sizeof(jobs[job_count].command) - 1);

jobs[job_count].command[
    sizeof(jobs[job_count].command) - 1
] = '\\0';
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session16_task2_jobtable.c

    job_count++;

    printf("[%d] %d started\n",
           jobs[job_count - 1].job_id,
           pid);
}

int main(void)
{
    char input[256];

    printf("Session 16 - Task 2: Job Table\n");
    printf("Commands:\n");
    printf("  run <command> - start background job\n");
    printf("  jobs          - display jobs\n");
    printf("  update        - update job states\n");
    printf("  cleanup       - remove completed jobs\n");
    printf("  exit          - quit\n\n");

    while (1)
    {
        update_jobs();

        printf("job-shell$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            break;
        }

        if (strcmp(input, "jobs") == 0)
        {
            update_jobs();
            display_jobs();
            continue;
        }

        if (strcmp(input, "update") == 0)
        {

```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session16_task2_jobtable.c
    }
    break;
}

if (strcmp(input, "jobs") == 0)
{
    update_jobs();
    display_jobs();
    continue;
}

if (strcmp(input, "update") == 0)
{
    update_jobs();
    printf("Job states updated.\n");
    continue;
}

if (strcmp(input, "cleanup") == 0)
{
    update_jobs();
    remove_completed_jobs();
    printf("Completed jobs removed.\n");
    continue;
}

if (strncmp(input, "run ", 4) == 0)
{
    char command[200];

    strcpy(command, input + 4);

    launch_background(command);
    continue;
}

printf("Unknown command.\n");
}

update_jobs();

printf("\nFinal Job Table:\n");
display_jobs();

return 0;
}
```